

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT Plan the learning first; let the checklist follow. Click a field to type • click a box to tick it.

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Studies (DigiChamps L10)	Grade 10 - Classes 10/1 to 10/6	Wk 13 (15-19 Nov 2026) - Period 1	Unit 4 - 4.2 Core Logic of the Game (1/2)
Pacing (2-1-1-2): ☒ On plan ☐ Ahead ☐ Behind Weeks ahead / behind: 0 Week / lesson: Week 13 / Lesson 1 of 2		Starting point (2-2-1-3): What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it? Countdowns working for ~80%; ms conversion now secure. Today: start the game, make worms pop out at random times in random holes.		

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — trace to the curriculum / standard and the table of specifications (ToS)

Students build core game logic: start the game, make worms pop out, set random times (Math.random range formula) and choose random holes. Sprint 4.2 obj. 1-4; CSTA 3A-AP-16; Activity 1.

Success criteria — what will successful learning look / sound like?

I start the game on a button click; I generate a random time between min and max; I choose a random hole; worms pop out repeatedly.

Key knowledge / skill / understanding:

Math.random() gives 0 to <1; min + rand*(max-min); Math.floor for whole numbers; array of holes.

Likely misconception / access or challenge need:

Math.random gives whole numbers; same worm can't appear twice in the same hole.

2 PROOF — What will students do to show they have learned it? Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 student task + evidence of learning	ACTIVITY 2 student task + evidence of learning	ACTIVITY 3 student task + evidence of learning
Strategy: Problem solving ☒ Time: 8 min	Strategy: Modelling / worked exam ☒ Time: 22 min	Strategy: Debate ☒ Time: 7 min
What students will do: Activity 1: calculate a random time between 2000 and 3000 ms for rand = 0.83 by hand, then verify in Chrome's Console.	What students will do: Code along: startGame(), randomTime(), randomHole() and popOut() using setTimeout; worms appear in random holes until the game ends.	What students will do: Is a game fair if it is random? Teams argue for/against using real examples (dice, sports draws) and suggest how to make randomness feel fair.
Evidence tool: Mini-whiteboards ☒ Assessment: ☒ Diagnostic ☐ Formative ☐ End Grouping: ☒ Individual ☐ Pair ☐ Group	Evidence tool: Practical / performance task ☒ Assessment: ☐ Diagnostic ☒ Formative ☐ End Grouping: ☒ Individual ☐ Pair ☐ Group	Evidence tool: Oral questioning ☒ Assessment: ☐ Diagnostic ☒ Formative ☐ End Grouping: ☐ Individual ☐ Pair ☒ Group
Resources: Chrome Console; formula slide.	Resources: VS Code; project source 'Core logic'; student book 4.2.	Resources: Debate prompt; examples.
Support (low performers / SEN): Calculator; worked example with 0.5.	Support (low performers / SEN): Code skeleton with functions named; pair programming.	Support (low performers / SEN): Sentence starters.
Stretch (fast learners / gifted): Write a function randomBetween(min,max) and test 5 values.	Stretch (fast learners / gifted): Prevent the same hole twice in a row (compare with lastHole).	Stretch (fast learners / gifted): Link to probability: chance of the same hole twice with 9 holes.

3 PREPARE — What do I need to do so students can succeed?				4 RESPOND — What will I do with what I see and hear?			
Set students up to succeed: ☒ Hook / prior learning ☒ Explain or model ☒ Success criteria shared ☒ Key vocabulary / examples ☒ Real-life connection ☒ Clear instructions / resources ☒ Praise, recognition and reinforcement planned				If students struggle or finish early I will: ☒ Re-teach ☒ Regroup ☒ Give extra support ☒ Stretch ☒ Have students redo / improve			
My setup / modelling: Time: 6 min Hook: roll a dice - how can a computer 'roll'? Model Math.random, the range formula, Math.floor with an array of holes. Show the flow: start -> random hole -> pop out -> hide -> repeat. Share success criteria.				What I will watch for / likely responses: Using Math.round giving index 9 (out of range); setTimeout not called again; CSS class for 'up' worm missing. Likely: 'worms appear all at once'.			
What is new or innovative in this lesson? <i>(ETEC looks for this)</i> Numeracy and probability embedded in coding; students debate fairness of randomness, linking maths to game design ethics.				Close / exit check <i>(summative + time for learners' questions)</i> Time: 7 min Exit ticket: compute min + 0.25*(max-min) for 1000-4000; why Math.floor not Math.round? Learners' questions.			
Progress recorded / shared: ☒ Gradebook ☐ Parents ☐ Written feedback in notebook / portfolio				Homework / platform <i>(e.g. Madrasati)</i> Madrasati: finish pop-out logic; upload a 20-second screen recording or screenshot. Optional: Challenge 3 Start/Stop random colour.			

5 REFLECT — After the lesson <i>Complete within a day. This is reflective-practice evidence and feeds your next Starting point.</i>		
What worked — and how do I know? Calculating by hand first made the code make sense; probability debate engaged quieter maths-strong students. 75% had worms popping by the end.	What will I change next time? Show the off-by-one error deliberately next time. Give a skeleton to all students to save typing time.	Students needing follow-up: Students whose worms don't hide (CSS class issue) - approx. 3 per class; peer mentors assigned for next lesson.

COVERAGE CHECK <i>Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.</i>							
Literacy read + write · 2-1-1-6 ☐ 1 ☐ 2 ☒ 3 ☐ C ☐ H	Numeracy numbers / data · 2-1-1-6 ☒ 1 ☐ 2 ☐ 3 ☒ C ☐ H	Digital students use it · 2-1-1-4 ☒ 1 ☒ 2 ☐ 3 ☐ C ☒ H	Digital research trusted sources · 2-1-1-4 ☐ 1 ☐ 2 ☐ 3 ☐ C ☒ H	Real-life ① application · 2-1-1-5 ☐ 1 ☐ 2 ☒ 3 ☐ C ☐ H	Real-life ② application · 2-1-1-5 ☒ 1 ☐ 2 ☐ 3 ☐ C ☐ H	Inquiry experiment / research · 2-1-1-7 ☒ 1 ☐ 2 ☐ 3 ☐ C ☐ H	Thinking problem-solving · 2-1-1-7 ☒ 1 ☒ 2 ☐ 3 ☐ C ☐ H
Create something new · 2-1-1-7 ☐ 1 ☒ 2 ☐ 3 ☐ C ☐ H	Discussion expressing ideas · 2-1-1-8 ☐ 1 ☐ 2 ☒ 3 ☐ C ☐ H	Pair / group cooperative work · 2-1-1-8 ☐ 1 ☐ 2 ☒ 3 ☐ C ☐ H	Learner choice independent work · 3-2-1-6 ☐ 1 ☐ 2 ☐ 3 ☐ C ☒ H	Self-assessment 2-2-1-1 ☐ 1 ☐ 2 ☐ 3 ☒ C ☐ H	Peer assessment 2-2-1-1 ☐ 1 ☐ 2 ☐ 3 ☐ C ☒ H	SEN / low specific support · 2-1-1-3 ☒ 1 ☒ 2 ☒ 3 ☐ C ☐ H	Gifted specific stretch · 2-1-1-3 ☒ 1 ☒ 2 ☒ 3 ☐ C ☐ H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback
--

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT Plan the learning first; let the checklist follow. Click a field to type • click a box to tick it.

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Studies (DigiChamps L10)	Grade 10 - Classes 10/1 to 10/6	Wk 13 (15-19 Nov 2026) - Period 2	Unit 4 - 4.2 Core Logic of the Game (2/2)
Pacing (2-1-1-2): ☒ On plan ☐ Ahead ☐ Behind Weeks ahead / behind: 0 Week / lesson: Week 13 / Lesson 2 of 2		Starting point (2-2-1-3): What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it? Exit ticket: 85% correct on range formula; Math.floor reason weaker. Today: scoring, tray, missed worms, results and edge cases.		

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — trace to the curriculum / standard and the table of specifications (ToS)

Students complete the game logic: score and worm count, add worms to the tray, calculate missed worms, display results with alert, return to index, use a gameOn variable and fix edge cases. Sprint 4.2 obj. 5-11; CSTA 3A-AP-21.

Success criteria — what will successful learning look / sound like?

My score increases only when a visible worm is clicked; missed worms = total - caught; results show at the end; clicks after game over do nothing.

Key knowledge / skill / understanding:

Variables store state; conditions check gameOn; edge cases are unusual inputs that break logic.

Likely misconception / access or challenge need:

Clicking fast counts multiple times is fine; testing once is enough.

2 PROOF — What will students do to show they have learned it? Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 student task + evidence of learning	ACTIVITY 2 student task + evidence of learning	ACTIVITY 3 student task + evidence of learning
Strategy: Modelling / worked exam ☐ Time: 12 min	Strategy: Problem solving ☐ Time: 18 min	Strategy: Cooperative learning ☐ Time: 7 min
What students will do: Code along: score and worm-count variables, click listener on worms, add worm image to tray, missed = total - caught.	What students will do: Implement results alert, back to index, and gameOn flag; then hunt edge cases (double click, click after end, empty name) and fix at least two.	What students will do: Playtest swap: pairs play another pair's game and log bugs in a shared sheet (bug, steps, expected, actual).
Evidence tool: Mini-whiteboards ☐ Assessment: ☐ Diagnostic ☒ Formative ☐ End Grouping: ☒ Individual ☐ Pair ☐ Group	Evidence tool: Practical / performance task ☐ Assessment: ☐ Diagnostic ☒ Formative ☐ End Grouping: ☐ Individual ☒ Pair ☐ Group	Evidence tool: Peer-assessment checklist ☐ Assessment: ☐ Diagnostic ☒ Formative ☐ End Grouping: ☐ Individual ☐ Pair ☒ Group
Resources: Project source; student book 4.2.	Resources: Edge-case card; browser console.	Resources: Shared Teams/Excel bug log.
Support (low performers / SEN): Skeleton code; checklist of variables.	Support (low performers / SEN): Edge-case card with hints for 2 cases.	Support (low performers / SEN): Bug log with one example row.
Stretch (fast learners / gifted): Store high score in sessionStorage.	Stretch (fast learners / gifted): Write a short test plan table with 5 cases and expected results.	Stretch (fast learners / gifted): Prioritise bugs by severity.

3 PREPARE — What do I need to do so students can succeed?				4 RESPOND — What will I do with what I see and hear?			
Set students up to succeed: ☒ Hook / prior learning ☒ Explain or model ☒ Success criteria shared ☒ Key vocabulary / examples ☒ Real-life connection ☒ Clear instructions / resources ☒ Praise, recognition and reinforcement planned				If students struggle or finish early I will: ☒ Re-teach ☒ Regroup ☒ Give extra support ☒ Stretch ☒ Have students redo / improve			
My setup / modelling: Time: 6 min Hook: a buggy version where score goes up when clicking empty holes. Model variables for state, the gameOn check, and how a test plan works. Explain playtesting roles.				What I will watch for / likely responses: Score updating on hidden worms; alert loops; gameOn never reset; students fixing bugs by deleting features. Response: 'it works on my computer' - test differently.			
What is new or innovative in this lesson? <i>(ETEC looks for this)</i> Professional playtesting with a shared bug log; students learn that software is tested by others before release.				Close / exit check <i>(summative + time for learners' questions)</i> Time: 7 min Exit check: write the formula for missed worms and one edge case you fixed. Learners' questions.			
Progress recorded / shared: ☒ Gradebook ☒ Parents ☒ Written feedback in notebook / portfolio				Homework / platform <i>(e.g. Madrasati)</i> Madrasati: fix one bug from your playtest log; upload before/after code screenshot.			

5 REFLECT — After the lesson <i>Complete within a day. This is reflective-practice evidence and feeds your next Starting point.</i>		
What worked — and how do I know? Bug log was a hit; students from different classes in later sections used the same format. 70% have a fully working game loop.	What will I change next time? Cut the modelling to 10 minutes; give more time to edge cases. Prepare a 'common bugs' slide.	Students needing follow-up: Students without a working game (approx. 5 per class): provide fixed source to continue; contact parents of 2 students with repeated missing work.

COVERAGE CHECK <i>Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.</i>							
Literacy read + write · 2-1-1-6 ☐ 1 ☐ 2 ☒ 3 ☐ C ☐ H	Numeracy numbers / data · 2-1-1-6 ☒ 1 ☐ 2 ☐ 3 ☒ C ☐ H	Digital students use it · 2-1-1-4 ☒ 1 ☒ 2 ☐ 3 ☐ C ☒ H	Digital research trusted sources · 2-1-1-4 ☐ 1 ☐ 2 ☐ 3 ☐ C ☒ H	Real-life ① application · 2-1-1-5 ☐ 1 ☐ 2 ☒ 3 ☐ C ☐ H	Real-life ② application · 2-1-1-5 ☐ 1 ☒ 2 ☐ 3 ☐ C ☐ H	Inquiry experiment / research · 2-1-1-7 ☐ 1 ☒ 2 ☐ 3 ☐ C ☐ H	Thinking problem-solving · 2-1-1-7 ☐ 1 ☒ 2 ☐ 3 ☐ C ☐ H
Create something new · 2-1-1-7 ☒ 1 ☐ 2 ☐ 3 ☐ C ☐ H	Discussion expressing ideas · 2-1-1-8 ☐ 1 ☐ 2 ☒ 3 ☐ C ☐ H	Pair / group cooperative work · 2-1-1-8 ☐ 1 ☒ 2 ☒ 3 ☐ C ☐ H	Learner choice independent work · 3-2-1-6 ☐ 1 ☐ 2 ☐ 3 ☐ C ☒ H	Self-assessment 2-2-1-1 ☐ 1 ☐ 2 ☐ 3 ☒ C ☐ H	Peer assessment 2-2-1-1 ☐ 1 ☐ 2 ☒ 3 ☐ C ☐ H	SEN / low specific support · 2-1-1-3 ☒ 1 ☒ 2 ☒ 3 ☐ C ☐ H	Gifted specific stretch · 2-1-1-3 ☒ 1 ☒ 2 ☒ 3 ☐ C ☐ H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback
--